OpenShift Serverless

Unified Multi-Cloud Serverless Platform

Cathay Technical Proposal | Validated on Live Infrastructure

Validated on ROSA HCP 4.20.21

2026-05-13 | Red Hat Adoption Team

Agenda

01 **Multi-Cloud Serverless Challenge**
N clouds x M functions = NxM independent CI/CD pipelines

02 **OpenShift Serverless Solution**
Knative Serving + Eventing + Functions — unified architecture

03 **Validated Comparison**
Knative vs Lambda: performance, complexity, and cost benchmarks

04 **Workflow Orchestration**
SonataFlow vs AWS Step Functions — live comparison

05 **Enterprise Capabilities**
Unified CI/CD, multi-cloud portability, event-driven architecture

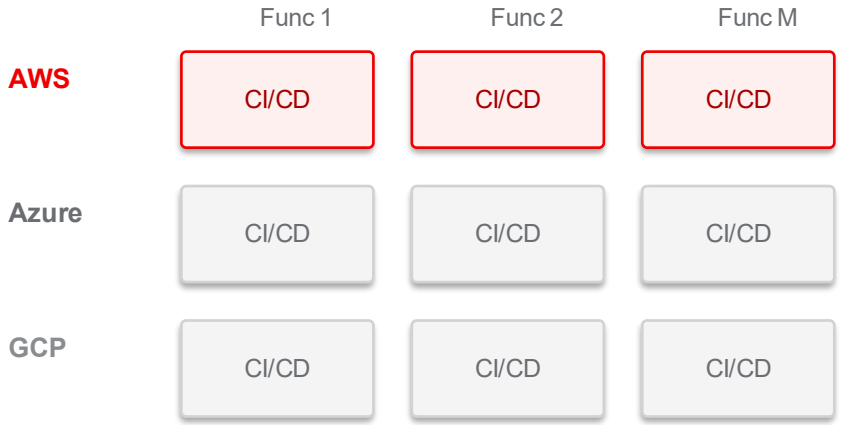
06 **Implementation Roadmap**
8-week phased rollout plan with success criteria

CHALLENGE

Multi-Cloud Serverless Challenge

N clouds x M functions = NxM independent CI/CD pipelines

- **Each cloud requires its own CI/CD pipeline**
- AWS Lambda: SAM/CDK + IAM + API Gateway
- Azure Functions: func CLI + bindings + triggers
- Alibaba Cloud / GCP: separate deployment tools & SDKs
- **Operational overhead scales linearly with cloud count**
- NxM pipelines = NxM maintenance cost
- Every change must be replicated across multiple platforms
- **Deep vendor lock-in**
- Proprietary SDKs, trigger models, and deployment tools
- Migration cost grows over time



3 clouds x 50 functions = 150 pipelines

Current State vs. Target State

From NxM pipelines to 1xM pipelines — architecture transformation

Current State

AWS Lambda
(multiple functions)

CI/CD for AWS
(SAM/CDK)

Azure Functions
(planned)

CI/CD for Azure
(func CLI)

N clouds x M functions = NxM pipelines

Migrate

Target State

Unified CI/CD Pipeline
(Tekton + ArgoCD)

Container Image (OCI) + Knative YAML

ROSA
(AWS)

ARO
(Azure)

OCP
(Ali/GCP)

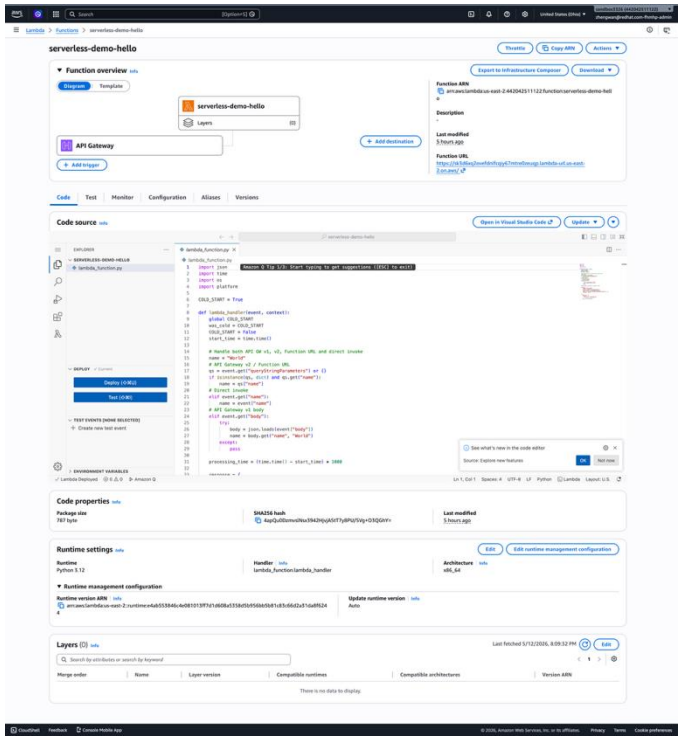
1 pipeline x M functions = M pipelines (cloud-agnostic)

VALIDATION

Test Environment

All data from end-to-end testing on live infrastructure

Component	Details
ROSA Cluster	rosa-fhmhp, Hosted CP, OCP 4.20.21, K8s v1.33.10
Worker Nodes	2x m6a.xlarge, us-east-2a
AWS Region	us-east-2 (Ohio)
Serverless Operator	v1.37.1 (Knative 1.17)
Serverless Logic Operator	v1.37.2 (GA, stable channel)
AWS Lambda Runtime	Python 3.12.13
Comparison Method	Same region, same network, same business logic



Performance: Knative Serving vs AWS Lambda

Same business logic, same AWS region, end-to-end measured data

Metric	AWS Lambda	OpenShift Serverless (Knative)	Notes
Cold Start Latency	~292ms	~1.67s	Lambda cold start is faster (no pod spin-up)
Warm Request Latency	~44-55ms	~28-30ms	Knative is faster (in-cluster network)
Scale-to-Zero	Auto (~15min)	Auto (~90s, configurable)	Knative scales down faster
Deployment Complexity	11 CLI commands 3 AWS services	1 oc apply 1 YAML	Knative greatly simplified
Multi-Cloud Support	AWS only	ROSA / ARO / OCP (any cloud)	Knative is portable
CI/CD Pipelines	Separate per cloud	One pipeline for all clusters	Ops cost N to 1

Warm latency: Knative 40% faster than Lambda | Deployment complexity reduced 90% | CI/CD pipelines: N to 1

Deployment Complexity: 11 Steps vs 1

Full steps required to create a simple HTTP function with an exposed endpoint

AWS Lambda

11 CLI commands | 3 AWS services

1. Write function code + ZIP package
2. Create IAM Trust Policy JSON
3. aws iam create-role
4. aws iam attach-role-policy
5. aws lambda create-function
6. aws lambda wait function-active
7. aws apigatewayv2 create-api
8. aws lambda add-permission
9. aws apigatewayv2 create-integration
10. aws apigatewayv2 create-route
11. aws apigatewayv2 create-stage

IAM + Lambda + API Gateway

OpenShift Serverless

1 command | 1 API

oc apply -f knative-service.yaml

- ✓ Route auto-generated (HTTPS)
- ✓ TLS certificate auto-configured
- ✓ Autoscaling policy via annotations
- ✓ No IAM, API Gateway, or permission setup
- ✓ No activation wait — ready in ~10s

Developer Experience: kn func vs Lambda

"Just write code, forget about containers" — Lambda-equivalent workflow

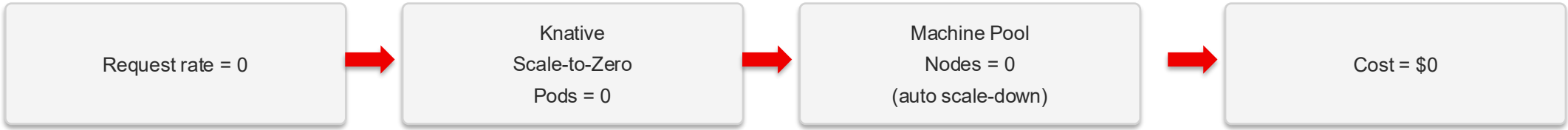
Step	AWS Lambda	kn func (Knative Functions)
Scaffold	sam init	kn func create -l python
Write Code	lambda_handler(event, context)	Function.handle(scope, receive, send)
Build	sam build / zip package	kn func build (S2I, no Dockerfile)
Deploy	sam deploy + IAM + API GW	kn service create --image ...
Dockerfile	Not needed	Not needed (S2I auto-build)
SDK Dependency	AWS SDK, boto3	None (standard HTTP/ASGI)
Languages	Python/Node/Go/Java/...	Python/Node/Go/Quarkus/Rust/TS
Warm Latency	~44-55ms	~28-31ms
Multi-Cloud	AWS only	Any OCP cluster

Also supports standard Dockerfile builds — same runtime, compatible with existing CI/CD pipelines

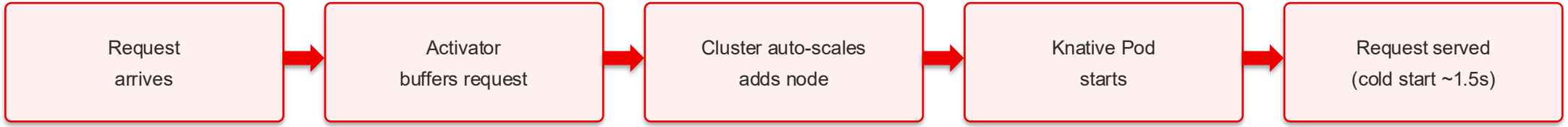
Scale-to-Zero: Automatic Cost Optimization

No traffic: Pods→0, Nodes→0, Compute cost→$0

No Traffic



Traffic Arrives



Scale-to-Zero Timeline

COST

Cost Comparison Model

Monthly cost estimates for a 50-function workload

Factor	AWS Lambda	Knative on ROSA (Spot)
Invocation Cost	$0.20/million requests	$0 (included in compute)
Compute (128MB, 1s)	$0.0000021/request	Scale-to-zero = $0 when idle
Warm-Up Cost	Provisioned Concurrency $$	minScale=0, no idle cost
Data Transfer	Cross-service charges	In-cluster = free
Multi-Cloud Overhead	N/A (single cloud)	Same image, same YAML
CI/CD Pipeline Cost	Separate per cloud $$	One pipeline for all
Node Idle Cost	N/A	$0 (auto scale-down removes nodes)

Traffic Scenario	AWS Lambda	Knative on ROSA (Spot)	Savings
Low (intermittent)	$200-500/mo	~$0 (fully scaled to zero)	100%
Medium	$500-2000+/mo	$200-600/mo	60-70%
Across 3 clouds	3x cost	1x CI/CD + 3x OCP compute	50%+

Note: For sustained high-throughput functions, Lambda may be more cost-effective. Knative excels for bursty/intermittent workloads and multi-cloud scenarios.

Event-Driven Architecture: Knative Eventing

Unified event model — replaces CloudWatch Events / SQS / S3 triggers

Mode 1: Direct Sink

PingSource → Service
Point-to-point, zero middleware

Current Demo

Mode 2: Channel + Sub

Source → Channel → Service A/B
Fan-out, ordered delivery

Medium

Mode 3: Broker + Trigger

Multi Source → Broker → Multi Service
Attribute-based routing

Production

Lambda Trigger	Knative Equivalent	Delivery Mode
CloudWatch Events / EventBridge	PingSource (cron schedule)	Direct Sink
SQS trigger	KafkaSource + AMQ Streams	Channel / Broker
S3 event	SinkBinding via Eventing	Broker + Trigger
API Gateway	Knative Route (auto-created)	HTTP direct
SNS	Knative Channel + Subscription	Channel

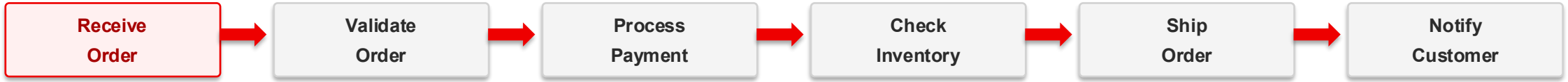
Design philosophy: Zero middleware for simple cases (Direct Sink); add Channel/Broker + Kafka only when needed

KEY REQUIREMENT

Workflow Orchestration: SonataFlow

Replaces AWS Step Functions — built on CNCF Serverless Workflow open standard

Order Processing Workflow — Live Demo



CNCF Serverless Workflow Spec

Open standard, not proprietary ASL — workflow definitions are runtime-portable

Kubernetes Native (CRD)

Workflow definitions are K8s CRs — seamless GitOps and CI/CD integration

jq Expression Engine

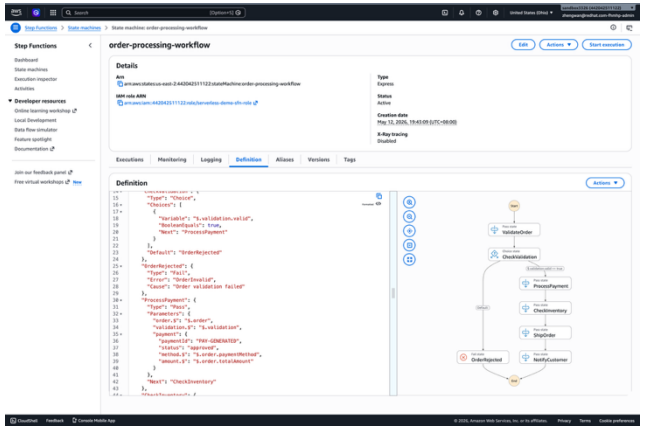
More powerful data transformation than ASL's JSONPath (Turing-complete)

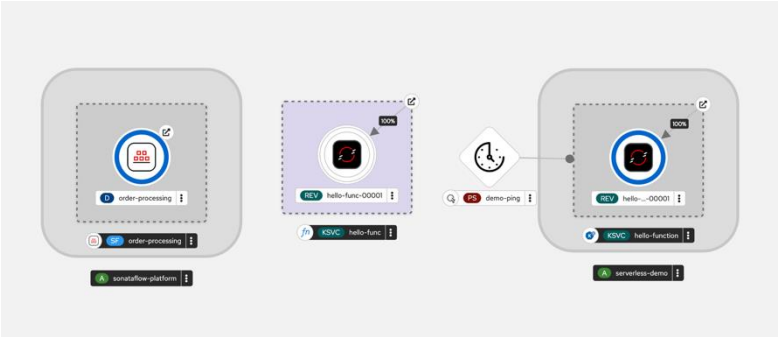
Built-in Dev Tools

SwaggerUI + DevUI out of the box, no additional configuration

Consistent Multi-Cloud Deploy

Same SonataFlow YAML deploys to ROSA, ARO, or any OCP cluster





SonataFlow vs AWS Step Functions

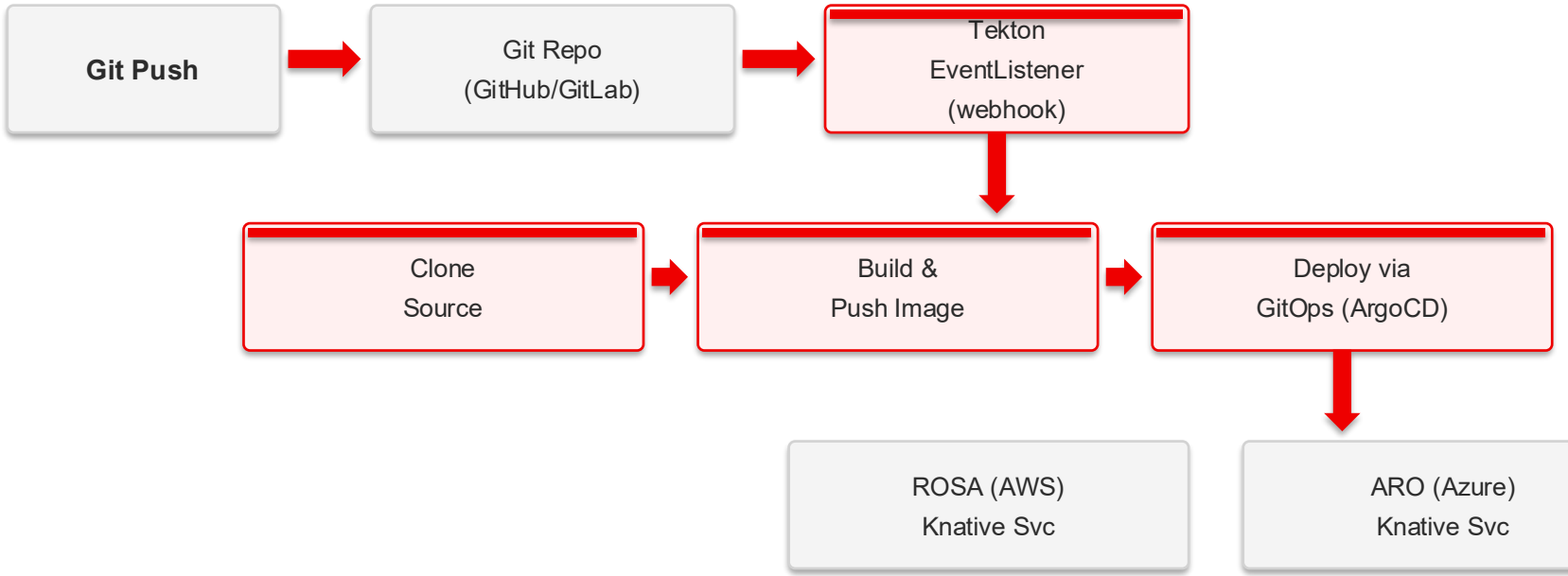
Same order-processing workflow, live comparison data

Dimension	AWS Step Functions	SonataFlow on ROSA
Workflow Execution Latency	~2ms (Express)	~46ms (warm request)
First Request	~962ms (incl. CLI overhead)	~220ms
Workflow Spec	ASL (AWS proprietary)	CNCF Serverless Workflow (open standard)
Expression Language	JSONPath + limited intrinsics	jq (full-featured, Turing-complete)
Deployment	aws sfn create-state-machine	oc apply -f sonataflow.yaml
Multi-Cloud Portable	AWS only	Any OCP cluster
K8s Native	No	Yes (CRD + Operator)
Human Approval	Activity Task + SQS	Built-in Callback State
Cost Model	Per state transition	Included in OCP compute
Visual Editor	Workflow Studio (built-in)	VS Code extension + Web Tools
GitOps Friendly	Console edits bypass Git	YAML CRD + Git version control
Vendor Lock-in	High (ASL proprietary)	Low (CNCF standard)

Design philosophy: AWS favors Console-first | SonataFlow favors Code-first + GitOps

Unified CI/CD: Tekton + ArgoCD

One pipeline for all OCP clusters — Git Push → Build → Deploy, fully automated



GitOps Repo Structure

base/
- knative-service-template.yaml

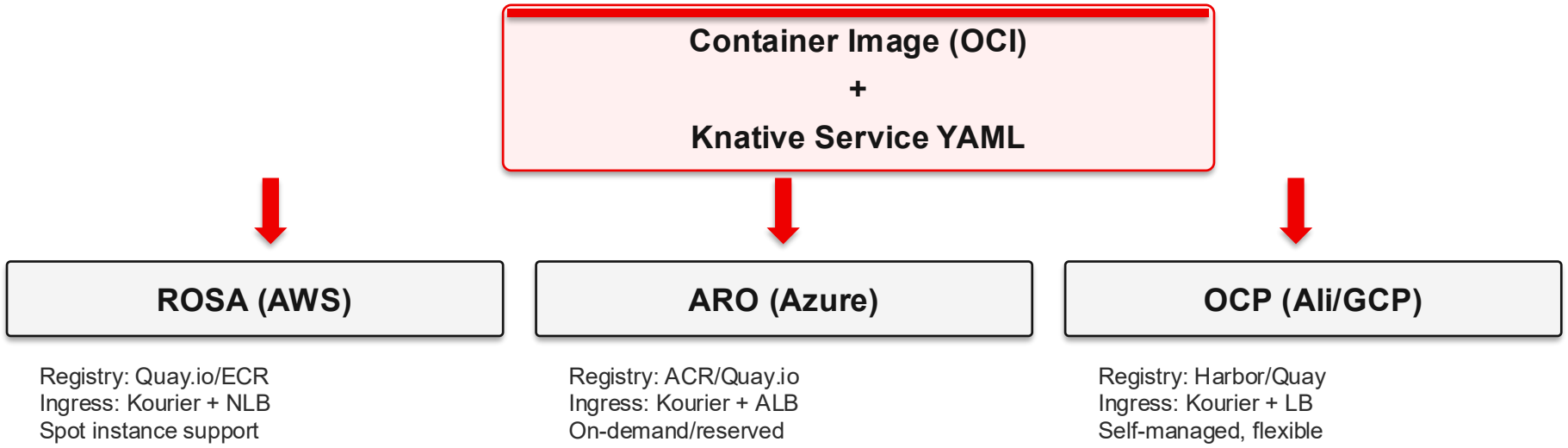
overlays/
- rosa-prod/ (scale, registry)
- aro-prod/ (registry override)
- ocp-ali/ (registry override)

argocd/
- rosa-prod-app.yaml
- aro-prod-app.yaml

Only differences: Registry URL + DNS + load balancer type — handled by Kustomize Overlays

Multi-Cloud Portability

Same container image + same Knative YAML → any OCP cluster



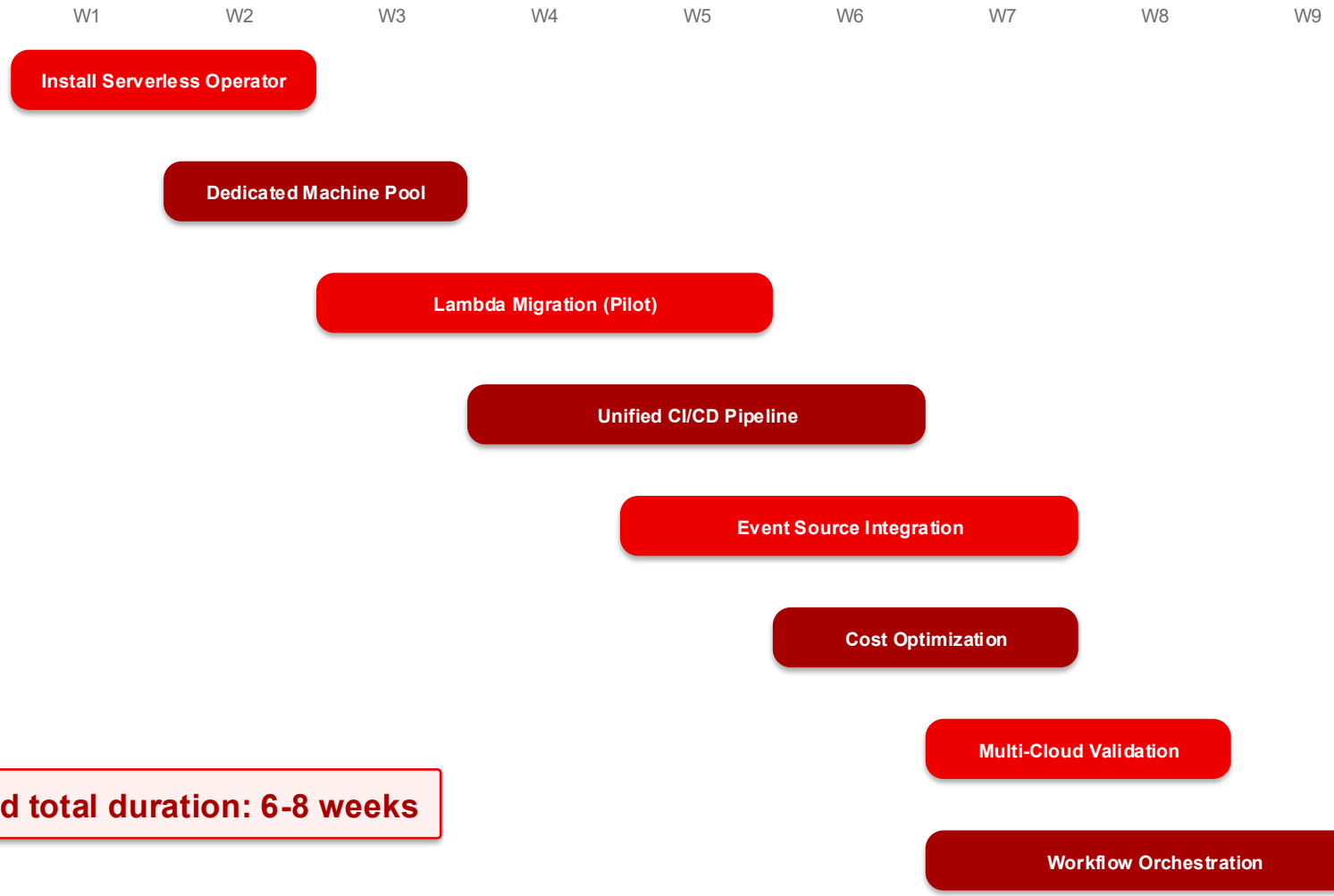
Lambda → Knative Concept Mapping

AWS Lambda Concept	Knative / OpenShift Equivalent
Lambda Function	Knative Service (ksvc)
API Gateway trigger	Knative Route (auto-created)
SQS trigger	KafkaSource + AMQ Streams
EventBridge Schedule	PingSource
Env variables	ConfigMap / Secret
Versioning / Alias	Knative Revisions + Traffic Splitting
SAM template	Knative YAML + Tekton Pipeline

ROADMAP

Implementation Roadmap

8-week phased rollout plan



Success Criteria

Measurable acceptance metrics

Criterion	Measurement	Test Status
Unified CI/CD	Single Tekton Pipeline deploys to all target clouds	Design ready
Scale-to-Zero	Pod count drops to 0, nodes removed	Validated ✓
Cost Reduction	Cost reduced ≥30% vs equivalent Lambda	Model validated
Cold Start < 10s	P95 cold start latency under 10 seconds	Measured 1.67s ✓
Multi-Cloud Portable	Same function image + YAML deploys to ROSA and ARO	Architecture ready
Migration Coverage	≥3 Lambda functions migrated as proof of concept	Pending
Ops Readiness	Monitoring dashboards, alerts, and runbook delivered	Docs ready
Workflow Orchestration	SonataFlow replaces Step Functions POC	Validated ✓

Key Takeaways

1. **Warm request latency: Knative 40% faster than Lambda**
   Measured: Knative ~28-30ms vs Lambda ~44-55ms — zero overhead from in-cluster networking

2. **Deployment complexity reduced from 11 steps to 1**
   No IAM, API Gateway, or permission setup — one declarative YAML deployment

3. **CI/CD pipelines reduced from N to 1**
   Tekton + ArgoCD + Kustomize Overlays → one pipeline for all clouds

4. **Scale-to-Zero delivers true pay-per-use**
   No traffic → Pods scale to zero → Nodes removed → Compute cost $0

5. **Built on open standards, eliminating vendor lock-in**
   Knative (CNCF) + CNCF Serverless Workflow + OCI container standards

Next step: Launch Phase 1 — install OpenShift Serverless on existing ROSA cluster, complete first pilot function migration within 2 weeks

Thank You

Q & A

Red Hat Adoption Team | 2026-05-13

Test Environment: ROSA HCP 4.20.21 (us-east-2)
Documentation: solution.md + steps.md (full commands and output)